BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 03

> **Objective & Lecture Mapping:** In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

> Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:
   `'grid_size': (self.width, self.height)`
   `'walls': list(self.walls)`
   `'all_food': list(self.food_positions)`

Step 1.2: Implementing BFS, DFS, and UCS

> You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

> The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space - The set of all possible states of a system and the connections between them (like a map - it exists once, with no duplicates).

Search Tree - A tree data structure used to locate specific keys from within a set efficiently — a tree of paths explored so far, where the same state can appear in multiple branches if reached by different routes.

So the state space is the "world," while the search tree is the algorithm's exploration record of that world, which can revisit the same state many times unless controlled.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

It prevents the search from re-expanding a state it has already visited. Without it, this changes from a 'Graph Search' back into a plain 'Tree Search', which can revisit the same cell endlessly, especially in a grid with cycles. Without 'reached', the DFS agent could loop forever bouncing between the same few cells (e.g., going back and forth around a wall), since nothing stops it from re-exploring states it already tried.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS expands nodes level by level (shortest path length first), so the first time it reaches the goal, that path is guaranteed to be the shortest possible (in steps), this matches uniform-cost grid movement, where each step costs the same. DFS instead dives as deep as possible down one branch before backtracking, with no regard for shortest distance, it just aims to reach any goal state, often finding a much longer, winding route before it happens to stumble onto the food.

4. (Evaluate) If we scaled `visual_grid_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity

formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS keeps every node at the current frontier level in memory - the number of nodes waiting in the queue grows exponentially with depth (O(b^d) space), so on a huge 1000x1000 grid, BFS's frontier could balloon to store thousands or millions of coordinates simultaneously, risking a memory crash. DFS only needs to store one path at a time down to the current depth (O(d) space) - far more memory-efficient, even though it may take a very long, inefficient path to actually find the food. So BFS trades memory for optimality, while DFS trades optimality for memory efficiency.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.